

CS-548 - Cloud-native Software Architectures

Assignment A4: Extending

Μάνος Αλέξανδρος csd-5136

Due Date: 20/04/2026

Άσκηση 1

Provide the YAML that allows you to manage a custom resource of type "Fruit" with Kubernetes.

(a) Install the custom resource.

Ακολουθώντας πιστά το [παράδειγμα](#) δημιουργούμε το αρχείο exerc1.yaml, πήρα και τα comments μαζί, και το κάνουμε "install" με το command:

kubectl apply -f exerc1.yaml

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc1$ kubectl apply -f exerc1.yaml
customresourcedefinition.apiextensions.k8s.io/fruits.hy548.csd.uoc.gr created
```

(b) Create the above instance.

Αποθηκεύουμε το instance στο αρχείο example.yaml και κάνουμε και δω:

kubectl apply -f example.yaml

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc1$ kubectl apply -f example.yaml
fruit.hy548.csd.uoc.gr/apple created
```

(c) Return the new instance in YAML format.

Τρέχουμε το command:

kubectl get fruit apple -o yaml

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc1$ kubectl get fruit apple -o yaml
apiVersion: hy548.csd.uoc.gr/v1
kind: Fruit
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"hy548.csd.uoc.gr/v1","kind":"Fruit","metadata":{"annotations":{},"name":"apple",
"namespace":"default"},"spec":{"count":3,"grams":980,"origin":"Krousonas"}}
  creationTimestamp: "2026-04-09T16:17:33Z"
  generation: 1
  name: apple
  namespace: default
  resourceVersion: "1058"
  uid: ee31772c-7ba0-425a-9104-1a961af6f3c1
spec:
  count: 3
  grams: 980
  origin: Krousonas
```

(d) Return a list of all available instances.

Για να πάρουμε όλη την λίστα τρέχουμε το command:

kubectl get fruits

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc1$ kubectl get fruits
NAME    AGE
apple   24m
```

Άσκηση 2

The example code for CRDs using the Python Kopf library is available on [GitHub](#).
Extend the example so that:

(a) The executable controller.py runs inside a container. Provide the Dockerfile. Build and upload the new container to Docker Hub.

Δημιουργούμε το docker file και το κάνουμε build to container με το command:
docker build -t nygma13/kopf-controller:v1 .

```
alexandros@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc2$ docker build -t nygma13/kopf-controller:v1 .
=> => sha256:a7730063fcfe708b222d34c4f07d633dfe087a28c05c4daaab2fa9943854c 49.30MB / 49.30MB 10.4s
=> => extracting sha256:a7730063fcfe708b222d34c4f07d633dfe087a28c05c4daaab2fa9943854c155 0.8s
=> => extracting sha256:f33970743aee750df25f6c661132b7401c8fefe930e5f4803f4c8b6f567a6b55 0.4s
=> => extracting sha256:5397da1d1537c4d725f3090c5688a582e14eeaf7743d75d9b38bad1649554987 1.2s
=> => extracting sha256:1ecbdc58afb44ec9b55fb51ec8bfdee5112cd31b39b564f5abb4567d094ffc 3.3s
=> => extracting sha256:c30fe4e5a0d64ddeb396947cd40be87429b302568458241d99054360e7d757cb 0.1s
=> => extracting sha256:0b4f6d60dcda26a8e3e7d6400d9d72a4a45228991b161cca93df7b35d594aeff 0.4s
=> => extracting sha256:4e8ce30c1c4c3682691cd17dc5be39717cb0f67dc289254b69341b3bce681167 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 70B 0.0s
=> [2/5] WORKDIR /app 2.6s
=> [3/5] COPY requirements.txt . 0.1s
=> [4/5] RUN pip install -r requirements.txt 18.3s
=> [5/5] COPY controller.py . 0.2s
=> exporting to image 5.2s
=> => exporting layers 3.7s
=> => exporting manifest sha256:6ea1dc226889361d5c62dbf2d2ba452c699d5d8c51d772cda602b71567b20 0.0s
=> => exporting config sha256:ea6194ac6b44fa2ae2c7e40b4b725e96ecc74579d039d052c6575434f5cb933 0.0s
=> => exporting attestation manifest sha256:4afdfdda42fe33059e0a592e8bc010110a4e3bb16941963f4 0.1s
=> => exporting manifest list sha256:43fc2f9197c7f535abc73d819a27df4fb58ac23410f318006385032f 0.0s
=> => naming to docker.io/nygma13/kopf-controller:v1 0.0s
=> => unpacking to docker.io/nygma13/kopf-controller:v1 1.2s
```

και το upload...

docker push nygma13/kopf-controller:v1

και μετά από λίγη ώρα:

```
alexandros@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc2$ docker push nygma13/kopf-controller:v1
The push refers to repository [docker.io/nygma13/kopf-controller]
f33970743aee: Pushed
c30fe4e5a0d6: Pushed
0b4f6d60dcda: Pushed
1ecbdc58afb: Pushed
f17ac4cdc343: Pushed
a7730063fcfe: Pushed
d02a0de3155e: Pushed
8c26ee4d7489: Pushed
4e8ce30c1c4c: Pushed
cb1b152ed7b8: Pushed
f341b8c59e52: Pushed
5397da1d1537: Pushed
v1: digest: sha256:43fc2f9197c7f535abc73d819a27df4fb58ac23410f318006385032fcc7ddd8e size: 856
```

(b) Provide greeting-controller.yaml, which will create a Deployment with the container you made. Make sure the necessary permissions are set so the controller can read the "Greeting" CRDs and create the corresponding Deployments with the hello-kubernetes container in all namespaces.

Όπως πριν, ακολουθούμε το [παράδειγμα](#) για την δημιουργία του greeting-controller.yaml και τρέχουμε τις εξής εντολές:

Create and Upload

```
docker build -t nygma13/kopf-controller:v1 .
```

```
docker push nygma13/kopf-controller:v1
```

Deploy and execute

```
kubectl apply -f greeting-crd.yaml
```

```
kubectl apply -f greeting-controller.yaml
```

```
kubectl apply -f hello-world.yaml
```

Provide the commands you used to verify that the deployment works correctly.

```
kubectl get pods -l app=exerc-controller
```

```
kubectl get deployments
```

```
kubectl get services
```

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc2$ kubectl get pods -l app=exerc-controller
NAME                                READY   STATUS    RESTARTS   AGE
exerc-controller-697858ff44-xg597   1/1     Running   0           78s
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc2$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
exerc-controller    1/1     1             1           5m51s
hello-to-all       3/3     3             3           26s
hello-world         2/2     2             2           26s
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc2$ kubectl get services
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
hello-to-all   ClusterIP   10.96.117.57 <none>        8080/TCP    31s
hello-world     ClusterIP   10.108.194.94 <none>        8080/TCP    31s
kubernetes      ClusterIP   10.96.0.1     <none>        443/TCP     139m
```

Άσκηση 3

(a) The executable controller.py runs inside a container. You will need to modify the code so that it uses SSL. Provide the Dockerfile. Build and upload the new container to Docker Hub.

Πρώτα ακολουθήσα τα βήματα που υπάρχουν στο README Start minikube and install cert-manager.

Μετά έκανα import την βιβλιοθήκη της ssl και πρόσθεσα το path όπου θα κάνει mount ο kubernetes στο function call app.run. Το dockerfile είναι σχετικά απλό, απλά κάνουμε expose το port. Μέτα build και push όπως στα προηγούμενα ερωτήματα.

```
docker build nygma13/webhook:v1 .
docker push nygma13/webhook:v1
```

(b) The webhook.yaml should use a service/deployment based on the new container instead of the Nginx proxy. Provide the new webhook.yaml.

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl get pods -n cert-manager
NAME                                READY   STATUS    RESTARTS   AGE
cert-manager-58f77bcf66-5jdz4       1/1     Running   0           102s
cert-manager-cainjector-6d49555c-j86bw 1/1     Running   0           102s
cert-manager-webhook-5575fc6dd-5tqwg 1/1     Running   0           102s
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl apply -f webhook.yaml
namespace/custom-label-injector unchanged
issuer.cert-manager.io/issuer-selfsigned created
certificate.cert-manager.io/controller-certificate created
service/controller unchanged
deployment.apps/controller unchanged
mutatingwebhookconfiguration.admissionregistration.k8s.io/custom-label-injector configured
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl get pods -n custom-label-injector
NAME                                READY   STATUS    RESTARTS   AGE
controller-67bd86b5c4-kn6r2         1/1     Running   0           13m
```

Από το webhook.yaml έκανα comment out τα configuration για το Nginx proxy (ConfigMap), έβαλα το custom image και ρύθμισα τον σωστό φάκελο στο mount (volumeMounts).

Μετά από κάποιες προσπάθειες (δεν έκανα save το .py):

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ docker push nygma13/webhook:v3
The push refers to repository [docker.io/nygma13/webhook]
a06f7ed51af0: Pushed
c30fe4e5a0d6: Layer already exists
8c26ee4d7489: Layer already exists
f33970743aee: Layer already exists
4e8ce30c1c4c: Layer already exists
ffac8260f14e: Layer already exists
0b4f6d60dcda: Layer already exists
25e97af8101c: Layer already exists
a7730063fcfe: Layer already exists
5397da1d1537: Layer already exists
1ecbddc58afb: Layer already exists
239fe28566b3: Layer already exists
v3: digest: sha256:6a4276bed129b549e7a9fb7c12c517abd2109d5651d46b6c5757d62d94d4f38e size: 856
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl apply -f webhook.yaml
namespace/custom-label-injector unchanged
issuer.cert-manager.io/issuer-selfsigned unchanged
certificate.cert-manager.io/controller-certificate configured
service/controller unchanged
deployment.apps/controller configured
mutatingwebhookconfiguration.admissionregistration.k8s.io/custom-label-injector configured
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl get pods -n custom-label-injector
NAME                                READY   STATUS    RESTARTS   AGE
controller-7874c744f-k2glk          1/1     Terminating    0           4m38s
controller-f9d468bc6-bph19          1/1     Running         0           7s
```

και για το testing:

```
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl run testing --image=nginx -n whatever
pod/testing created
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl get testing -n whatever --show-labels
error: the server doesn't have a resource type "testing"
alexandrosm@naxi:~/Desktop/csd/8th_sem/hy-548/Assignment4/exerc3$ kubectl get pod testing -n whatever --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
testing   1/1     Running   0           21s   custom-label=true,run=testing
```